



EA551

Gerenciamento de processos em um sistema multiprogramado.

Ana Carolina Vieira Araújo 248734

Índice

1 EA551 DataLoader: **2**
1.1 Mudanças no train.py: . . . 2
1.2 shm$_c$*ommon.h* : . . . 3
1.3 producer.c: . . . 3
1.4 Resultados e análise: . . . 6

2 Multiplicação de matrizes otimizada: **7**
2.1 Principais mudanças: . . . 7
2.2 matmul.h . . . 8
2.3 matmul.c . . . 8
2.4 Script cálculo diferença de tempo entre as versões não otimizada e otimizada: . . . 12
2.5 Resultados: . . . 14

3 Conversor .npy -> .emat: **14**

1 EA551 DataLoader:

1.1 Mudanças no train.py:

```
import posix_ipc
from multiprocessing import shared_memory
import struct

SEM_EMPTY_NAME = "/sem_empty"
SEM_FULL_NAME = "/sem_full"
SEM_MUTEX_NAME = "/sem_mutex"
SHM_NAME = "/shm_mnist"

IMAGE_SIZE = 28 * 28
SAMPLE_SIZE = (IMAGE_SIZE * 4) + 4  # Float32 (784 * 4 bytes) + Label Int32 (4
    bytes)
BUFFER_SIZE = 1000

sem_empty = posix_ipc.Semaphore(SEM_EMPTY_NAME)
sem_full = posix_ipc.Semaphore(SEM_FULL_NAME)
sem_mutex = posix_ipc.Semaphore(SEM_MUTEX_NAME)

shm = shared_memory.SharedMemory(name=SHM_NAME.lstrip('/'))
```

Essa parte é responsável por criar a ponte de comunicação entre o programa escrito em Python e a memória compartilhada do sistema operacional (criada previamente pelo programa em C). Em vez de ler arquivos do disco rígido a cada segundo, o Python acessa diretamente a memória RAM do computador para pegar os dados instantaneamente.

Para garantir que essa conversa entre o C e o Python ocorra sem erros, o código importa bibliotecas especializadas e define identificadores globais com nomes iniciados por uma barra, como /sem_empty e /shm_mnist. Esses nomes funcionam como "endereços"no sistema operacional. O programa em Python usa esses endereços para localizar exatamente onde os semáforos e a memória compartilhada estão guardados na memória RAM.

Em seguida, o código calcula o tamanho exato de cada imagem na memória. Ele define que cada amostra do MNIST tem 784 pixels (28×28) e que, ao converter esses números e incluir a identificação do dígito (o rótulo de 0 a 9), cada imagem ocupa exatamente 3.140 bytes de memória. Também é definido um buffer com capacidade para armazenar até 1.000 imagens simultaneamente na RAM, garantindo que sempre haja dados prontos para o modelo de inteligência artificial processar sem ter que esperar.

Por fim, o script se conecta aos três semáforos de controle (sem_empty, sem_full e sem_mutex) e ao bloco de memória compartilhada /dev/shm. Os semáforos servem para sincronizar a leitura. Com isso, os dados trafegam direto pela memória RAM.

```
sem_full.acquire()
sem_mutex.acquire()
```

```
3
4  sample_offset = read_idx * SAMPLE_SIZE
5
6  img_bytes = bytes(shm.buf[sample_offset : sample_offset + (IMAGE_SIZE * 4)])
7  label_bytes = bytes(shm.buf[sample_offset + (IMAGE_SIZE * 4) : sample_offset +
       SAMPLE_SIZE])
8
9  image_np = np.frombuffer(img_bytes, dtype=np.float32).reshape(1, 1, 28, 28)
10 image_tensor = torch.from_numpy(image_np)
11 image_class = struct.unpack("i", label_bytes)[0]
12
13 read_idx = (read_idx + 1) % BUFFER_SIZE
14
15 sem_mutex.release()
16 sem_empty.release()
```

Dentro do laço principal de treinamento, a extração de dados da memória compartilhada é comandada pela sincronização POSIX. Inicia a leitura executando sem_full.acquire(), o que congela a execução do consumidor até que o produtor em C disponibilize uma nova amostra no ring buffer. Em seguida, o semáforo sem_mutex é travado para garantir acesso exclusivo à região crítica e evitar condições de corrida durante o cálculo do deslocamento de memória.

A partir desse ponto, o código lê o bloco contínuo e gera cópias locais dos vetores brutos via bytes(...). Essa etapa é fundamental para desacoplar a memória do Python do ponteiro original da RAM, permitindo que os semáforos sem_mutex e sem_empty sejam liberados imediatamente para que o programa em C volte a preencher o slot vago.

Por fim, os bytes da imagem são mapeados de forma eficiente para matrizes do NumPy e convertidos em tensores do PyTorch, enquanto o rótulo do dígito é decodificado via struct.unpack, garantindo que os dados cheguem à rede neural com o mínimo de sobrecarga de processamento.

1.2 $\mathbf{shm}_{c}ommon.h$:

Esse arquivo de cabeçalho (shm_common.h) estabelece a estrutura de dados compartilhada na memória RAM entre o produtor em C e o consumidor em Python. Ele utiliza diretivas de proteção (include guards) e a biblioteca <stdint.h> para garantir a consistência dos tipos de dados binários no kernel Linux. O código define os nomes dos recursos POSIX no diretório /dev/shm e configura um ring buffer estático com capacidade para 256 amostras de 784 pixels (28×28). A estrutura Sample agrupa os 3.136 bytes dos pixels em ponto flutuante ao rótulo inteiro do dígito, enquanto a estrutura SharedBuffer encapsula o vetor circular de dados, os índices de leitura e escrita (read_idx e write_idx) e uma flag de terminação (is_done).

1.3 producer.c:

```
1  void collect_files(const char *base_path) {
2      for (int digit = 0; digit <= 9; digit++) {
3          char dir_path[1024];
```

```
        snprintf(dir_path, sizeof(dir_path), "%s/%d", base_path, digit);

        DIR *dir = opendir(dir_path);
        if (!dir) continue;

        struct dirent *entry;
        while ((entry = readdir(dir)) != NULL) {
            if (strstr(entry->d_name, ".emat")) {
                file_tasks = realloc(file_tasks, sizeof(FileTask) * (
    total_files + 1));
                snprintf(file_tasks[total_files].filepath, sizeof(file_tasks[
    total_files].filepath), "%s/%s", dir_path, entry->d_name);
                file_tasks[total_files].label = digit;
                total_files++;
            }
        }
        closedir(dir);
    }
}
```

A função collect_files realiza o mapeamento sequencial e indexação prévia de toda a base de dados em disco. Navegando iterativamente pelas subpastas numéricas de 0 a 9 do diretório raiz, abre cada pasta e varre seus arquivos via readdir, filtrando exclusivamente aqueles com extensão binária .emat. Para cada arquivo válido encontrado, a função utiliza realloc para expandir dinamicamente o vetor de estruturas file_tasks na RAM, armazenando o caminho absoluto do arquivo e o seu rótulo numérico. Essa etapa de varredura prévia constrói a lista global de tarefas que será posteriormente dividida e processada em paralelo pelas threads do produtor em C.

```
void *worker_thread(void *arg) {
    while (1) {
        size_t idx;
        pthread_mutex_lock(&task_mutex);
        if (current_file_idx >= total_files) {
            pthread_mutex_unlock(&task_mutex);
            break;
        }
        idx = current_file_idx++;
        pthread_mutex_unlock(&task_mutex);

        int fd = open(file_tasks[idx].filepath, O_RDONLY);
        if (fd < 0) continue;

        EmatHeader header;
        if (read_all(fd, &header, sizeof(EmatHeader)) != sizeof(EmatHeader)) {
            close(fd);
            continue;
        }

        float temp_pixels[IMAGE_SIZE];
        if (header.dtype == 0) { // UINT8
            uint8_t raw[IMAGE_SIZE];
```

```
            read_all(fd, raw, IMAGE_SIZE);
            for (int i = 0; i < IMAGE_SIZE; i++) temp_pixels[i] = (float)raw[i
    ];
        } else if (header.dtype == 2) { // FLOAT32
            read_all(fd, temp_pixels, sizeof(float) * IMAGE_SIZE);
        }
        close(fd);

        sem_wait(sem_empty);
        sem_wait(sem_mutex);

        int slot = shm_ptr->write_idx;
        memcpy(shm_ptr->samples[slot].pixels, temp_pixels, sizeof(float) *
    IMAGE_SIZE);
        shm_ptr->samples[slot].label = file_tasks[idx].label;
        shm_ptr->write_idx = (shm_ptr->write_idx + 1) % BUFFER_SIZE;

        sem_post(sem_mutex);
        sem_post(sem_full);
    }
    return NULL;
}
```

As threads obtêm as imagens a processar pelo vetor global de tarefas file_tasks. A variável current_file_idx é protegida pelo task_mutex para evitar condições de corrida. A thread tranca o mutex, copia o índice do arquivo atual, incrementa o contador global e destranca-o de imediato, minimizando o tempo de bloqueio entre os processos. Se todas as amostras já tiverem sido atribuídas, o laço é encerrado.

A leitura dos pixels adapta-se ao tipo de dado especificado em header.dtype. Se os dados estiverem em UINT8, os 784 bytes são lidos e convertidos para float de 32 bits. Se já estiverem em FLOAT32, a leitura é feita diretamente para o vetor temporário temp_pixels.

Por fim, a thread submete-se aos semáforos. Executa sem_wait(sem_empty) para aguardar uma vaga no buffer e tranca o sem_mutex para ir à memória compartilhada. Copia a imagem e o rótulo com memcpy para a posição write_idx, atualiza o índice circular, liberta o sem_mutex e sinaliza o consumidor Python através de sem_post(sem_full).

```
int main(int argc, char *argv[]) {
    int num_threads = (argc > 1) ? atoi(argv[1]) : 1;
    const char *emat_dir = (argc > 2) ? argv[2] : "../mnist_emat/train";

    printf("Indexando arquivos .emat de %s...\n", emat_dir);
    collect_files(emat_dir);
    printf("Total de amostras encontradas: %zu\n", total_files);

    shm_unlink(SHM_NAME);
    sem_unlink(SEM_EMPTY_NAME);
    sem_unlink(SEM_FULL_NAME);
    sem_unlink(SEM_MUTEX_NAME);
```

```
13
14      int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
15      if (ftruncate(shm_fd, sizeof(SharedBuffer)) == -1) {
16          perror("ftruncate");
17          return 1;
18      }
19      shm_ptr = mmap(0, sizeof(SharedBuffer), PROT_READ | PROT_WRITE, MAP_SHARED
        , shm_fd, 0);
20
21      memset(shm_ptr, 0, sizeof(SharedBuffer));
22
23      sem_empty = sem_open(SEM_EMPTY_NAME, O_CREAT, 0666, BUFFER_SIZE);
24      sem_full  = sem_open(SEM_FULL_NAME, O_CREAT, 0666, 0);
25      sem_mutex = sem_open(SEM_MUTEX_NAME, O_CREAT, 0666, 1);
26
27      pthread_t threads[num_threads];
28      for (int i = 0; i < num_threads; i++) {
29          pthread_create(&threads[i], NULL, worker_thread, NULL);
30      }
31
32      for (int i = 0; i < num_threads; i++) {
33          pthread_join(threads[i], NULL);
34      }
35
36      shm_ptr->is_done = 1;
37      sem_post(sem_full);
38
39      printf("Produção concluída pelo C.\n");
40      return 0;
41  }
```

1.4 Resultados e análise:

Métrica de Desempenho	**Baseline (Python + .txt)**	**Proposto (C + SHM + .emat)**
Tamanho do Dataset	60.000 amostras	60.000 amostras
Tempo Total de Execução	~ 448 s ($\sim 7,5$ min)	~ 18 s
Vazão de Processamento	~ 134 it/s	$\mathbf{3.317,77}$ it/s
Proporção de Tempo em I/O (`read_pct`)	$61,2\%$	$\mathbf{7,3}\%$
Acurácia de Classificação	$99,9\%$	$99,95\%$

Configuração	**Tempo Total (s)**	**Vazão (it/s)**	**`read_pct` (%)**	**Acurácia (%)**	***Speedup***
Baseline (Python puro / .txt)	$\sim 448,0$ s	$\sim 134,00$ it/s	$61,2\%$	$99,95\%$	$1,00\times$
Produtor C (1 Thread)	$14,0$ s	$4.219,72$ it/s	$7,1\%$	$99,95\%$	$32,00\times$
Produtor C (2 Threads)	$14,0$ s	$4.170,10$ it/s	$7,1\%$	$99,95\%$	$32,00\times$
Produtor C (4 Threads)	$13,0$ s	$4.295,89$ it/s	$7,0\%$	$99,95\%$	$34,46\times$
Produtor C (8 Threads)	$13,0$ s	$4.290,06$ it/s	$7,0\%$	$99,95\%$	$34,46\times$

Os resultados demonstram mudança no regime de execução do pipeline de treino. No modelo original em Python, a leitura síncrona dos ficheiros de texto representava o principal gargalo do

sistema, consumindo 61,2% do tempo total apenas em operações de leitura. A implementação da arquitetura em C com memória partilhada reduziu essa sobrecarga para cerca de 7,0%, acelerando o processo. A taxa de processamento saltou de cerca de 134 para mais de 4.200 imagens por segundo, reduzindo o tempo total de execução de aproximadamente 7,5 minutos para apenas 13 segundos, o que equivale a um ganho de velocidade de mais de 34 vezes.

A variação no número de threads do produtor em C mostra que o ponto de saturação é atingido precocemente. A utilização de apenas uma thread já foi suficiente para proporcionar um ganho de 32 vezes na velocidade em relação ao modelo inicial. Ao aumentar o número de threads para 2, 4 ou 8, o tempo total estabiliza nos 13 a 14 segundos. Este comportamento ocorre porque o produtor em C passa a preencher o buffer na memória RAM muito mais rápido do que o PyTorch consegue processar, transferindo o limite do pipeline da leitura de disco para a capacidade de computação da CPU no treino da rede neural. Por fim, a manutenção da acurácia em 99,95% em todos os cenários confirma a integridade total dos dados transferidos.

2 Multiplicação de matrizes otimizada:

2.1 Principais mudanças:

O Lab 5 foi reformulado para transformar um algoritmo simples de multiplicação de matrizes em uma solução paralela. A principal mudança em relação à versão anterior está na forma como organizamos a memória e o processamento, que ao invés de percorrer matrizes fragmentadas, agora dividimos a memória em um bloco único e contínuo na RAM e repartimos o trabalho entre vários núcleos do processador.

A execução começa na função multiply_matrices, que confere se a multiplicação é possível e cria a matriz resultado C. Diferente do modelo anterior, onde cada linha ficava espalhada na memória, agora toda a matriz fica junta em um único bloco. Para dividir o trabalho sem deixar nenhuma thread sobrecarregada, o código calcula quantas linhas cada uma vai processar e distribui o resto da divisão, dando uma linha a mais para as primeiras threads.

Em seguida, função multiply_work em vez de calcular a matriz inteira de uma vez e lotar a memória, usamos tiling para fatiar o problema em blocos pequenos de 64x64. Somado a isso, reordenamos os laços para i, k, j e guardamos valores frequentes em registradores locais da CPU. Essa combinação faz com que o processador leia os dados sempre em linha reta na memória e mantenha tudo dentro da memória Cache L1, que é super rápida.

Por fim, a thread principal usa o pthread_join() como uma barreira, esperando que todas as workers terminem suas tarefas para garantir que a matriz C esteja completa.

Para fechar, a função compare_matrices valida se o resultado está correto. Ela compara a matriz calculada com uma versão de referência elemento por elemento, usando uma margem de tolerância para ignorar diferenças de arredondamento em ponto flutuante causadas pela reordenação das somas.

2.2 matmul.h

```
typedef struct {
    uint32_t rows;
    uint32_t cols;
    double *data;
} Matrix;
```

Mudança de double ** para double *, para alocação de uma matriz unidimensional contígua.

2.3 matmul.c

```
#!/bin/bash

#define NUM_THREADS 4
#define TILE_SIZE 64

typedef struct {
    const Matrix *A;
    const Matrix *B;
    Matrix *C;
    uint32_t start_row;
    uint32_t end_row;
} ThreadData;
```

Primeiramente, é criada uma struct para uma Thread. Ela conterá um ponteiro para a matriz A, um para a matriz B e outra para a C. Além da indicação da linha inicial e linha final que a thread irá executar.

Usaremos essa struct posteriormente para fazer a divisão do cálculo da matriz para haver paralelismo entre as diferentes threads, assim distribuindo o trabalho de forma equilibrada e sem conflitos de escrita.

Vamos usar 4 threads para a divisão, e dividir cada bloco em submatrizes de 64x64, que vai ocupar 32k bytes, e espera-se que como a cache L1 geralmente tem esse tamanho nos processadores, vamos poder reduzir o acesso à memória RAM e fazer com que essas operações ocorram de maneira mais rápida ainda.

```
Matrix *allocate_matrix(uint32_t rows, uint32_t cols) {
    Matrix *mat = (Matrix *)malloc(sizeof(Matrix));
    if (!mat) return NULL;

    mat->rows = rows;
    mat->cols = cols;

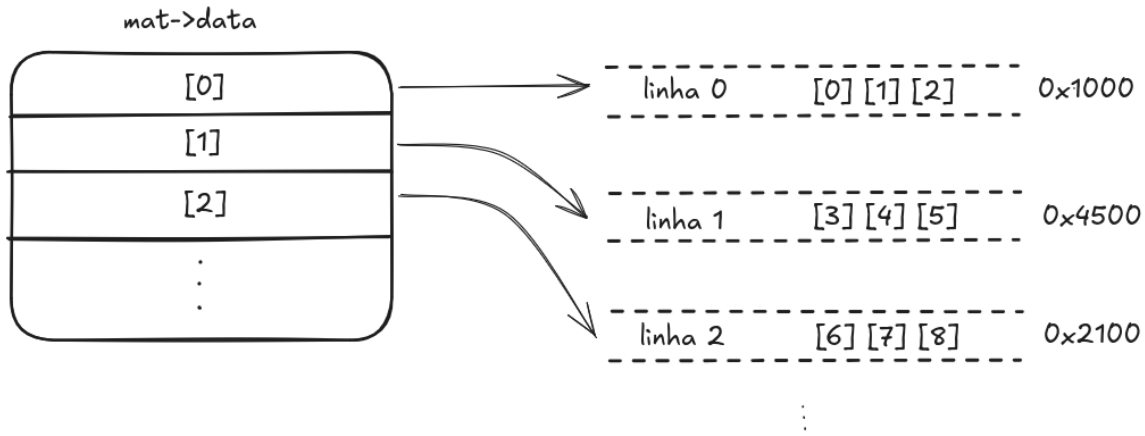
    mat->data = (double **)malloc(rows * sizeof(double *));
    if (!mat->data) {
        free(mat);
```

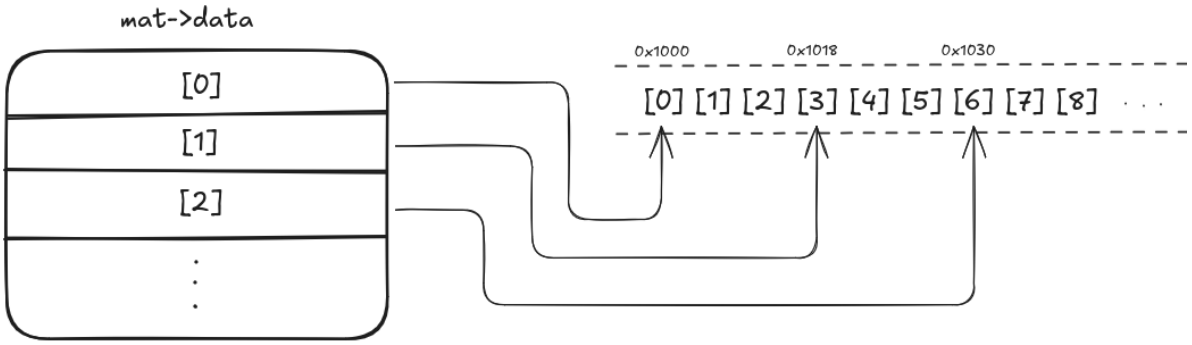
```
        return NULL;
    }

    double *block = (double *)calloc((size_t)rows * cols, sizeof(double));
    if (!block) {
        free(mat->data);
        free(mat);
        return NULL;
    }

    for (uint32_t i = 0; i < rows; i++) {
        mat->data[i] = &block[i * cols];
    }

    return mat;
}
```





Enquanto o Laboratório 4 alocava cada linha de forma independente por múltiplos malloc(), gerando blocos dispersos na RAM, o Laboratório 5 utiliza um único bloco contínuo para todos os elementos via calloc(), mantendo os ponteiros mat->data[i] mapeados para os offsets correspondentes (&block[i * cols]). Essa continuidade física maximiza a localidade espacial, eliminando gargalos de acesso à RAM. Além disso, essa estrutura otimiza a função read_emat, possibilitando o preenchimento sequencial dos dados do arquivo diretamente no bloco único de memória

em poucas operações de E/S.

```
void *multiply_worker(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    const Matrix *A = data->A;
    const Matrix *B = data->B;
    Matrix *C = data->C;

    uint32_t M = data->end_row;
    uint32_t K = A->cols;
    uint32_t N = B->cols;

    // Loop Tiling + Reordenação i -> k -> j
    for (uint32_t i0 = data->start_row; i0 < M; i0 += TILE_SIZE) {
        for (uint32_t k0 = 0; k0 < K; k0 += TILE_SIZE) {
            for (uint32_t j0 = 0; j0 < N; j0 += TILE_SIZE) {

                uint32_t i_max = (i0 + TILE_SIZE < M) ? i0 + TILE_SIZE : M;
                uint32_t k_max = (k0 + TILE_SIZE < K) ? k0 + TILE_SIZE : K;
                uint32_t j_max = (j0 + TILE_SIZE < N) ? j0 + TILE_SIZE : N;

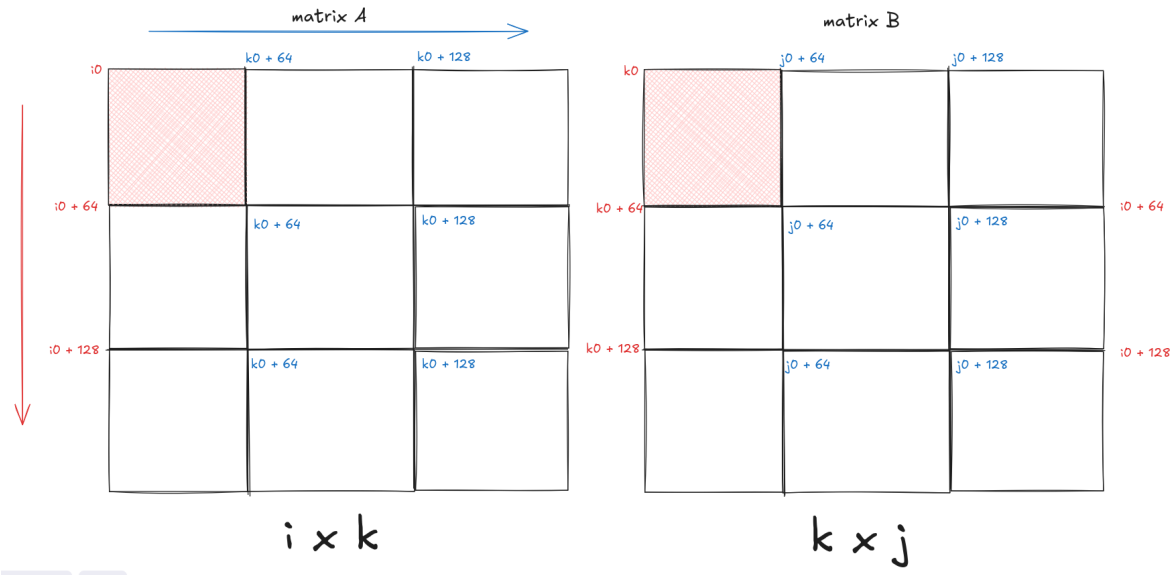
                for (uint32_t i = i0; i < i_max; i++) {
                    for (uint32_t k = k0; k < k_max; k++) {
                        double r = A->data[i][k];
                        for (uint32_t j = j0; j < j_max; j++) {
                            C->data[i][j] += r * B->data[k][j];

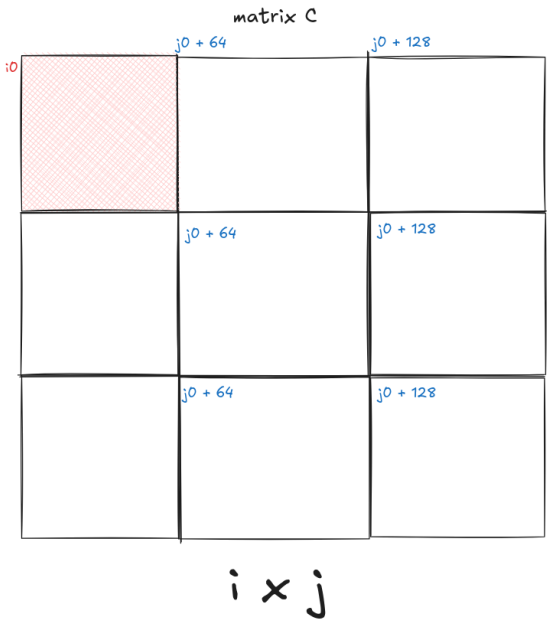

    pthread_exit(NULL);
}
```

Agora, é feita a divisão das tiles. pela estrutura da thread são definidos a linha que a thread vai processar, o número de colunas da matriz A e o da B/C.

Após é feita a divisão de 64x64 dos blocos, com o for incrementando em TILE_SIZE as linhas/colunas. O tratamento de bordas para o caso de matrizes não multiplas de 64 também é feito.

Após essas definições o processo de multiplicação ocorre da mesma maneira do anterior.






```c
Matrix *multiply_matrices(const Matrix *A, const Matrix *B) {
    if (A->cols != B->rows) return NULL;

    Matrix *C = allocate_matrix(A->rows, B->cols);
    if (!C) return NULL;

    pthread_t threads[NUM_THREADS];
    ThreadData thread_data[NUM_THREADS];

    uint32_t rows_per_thread = A->rows / NUM_THREADS;
    uint32_t extra_rows = A->rows % NUM_THREADS;
    uint32_t current_row = 0;

    for (int t = 0; t < NUM_THREADS; t++) {
```

```
        thread_data[t].A = A;
        thread_data[t].B = B;
        thread_data[t].C = C;
        thread_data[t].start_row = current_row;

        thread_data[t].end_row = current_row + rows_per_thread + (t < (int)
    extra_rows ? 1 : 0);
        current_row = thread_data[t].end_row;

        if (pthread_create(&threads[t], NULL, multiply_worker, &thread_data[t
    ]) != 0) {
            perror("Erro ao criar thread");
            free_matrix(C);
            return NULL;
        }
    }

    for (int t = 0; t < NUM_THREADS; t++) {
        pthread_join(threads[t], NULL);
    }

    return C;
}
```

Aqui está uma sugestão de texto explicativo em um único parágrafo para colocares no relatório:

Atua como gerenciadora da paralelização, sendo responsável por validar as dimensões das matrizes de entrada e alocar a matriz resultante C. Para garantir o balanceamento de carga, a função divide o total de linhas pelo número de threads e distribui o resto da divisão de forma entre as primeiras. Em seguida, os ponteiros das matrizes e os limites de linhas são empacotados na ThreadData e passados para a criação das workers. Por fim, a main thread executa o pthread_join(), funcionando como uma barreira de sincronização que aguarda o término de todas as threads antes de retornar a matriz C totalmente calculada.

2.4 Script cálculo diferença de tempo entre as versões não otimizada e otimizada:

```
#!/bin/bash

EXEC_LAB4="../../LABORATÓRIO4/arquivos_atividade_extra_classe/matmul"
EXEC_LAB5="./matmul"
TESTES_DIR="testes_4.3"

echo "============================================================"
echo "    BENCHMARK COMPARATIVO: LAB 4 vs LAB 5 (Otimizado)"
echo "============================================================"
printf "%-10s | %-15s | %-15s | %-10s\n" "Teste" "Lab 4 (s)" "Lab 5 (s)" "
   Speedup"
echo "------------------------------------------------------------"

```

```
for i in {1..3}; do
    FILE_A="${TESTES_DIR}/A_${i}.emat"
    FILE_B="${TESTES_DIR}/B_${i}.emat"
    FILE_C="${TESTES_DIR}/C_${i}.emat"
    ARGS="$FILE_A $FILE_B $FILE_C"

    if [ -f "$EXEC_LAB4" ]; then
        TIME_LAB4=$({ time $EXEC_LAB4 $ARGS > /dev/null; } 2>&1 | grep real | grep -oE '[0-9]+\.[0-9]+' | tail -n 1)
    else
        TIME_LAB4="N/A"
    fi

    TIME_LAB5=$( $EXEC_LAB5 $ARGS 2>&1 | grep "Tempo de execução" | grep -oE '[0-9]+\.[0-9]+' )

    if [ -z "$TIME_LAB5" ]; then TIME_LAB5="N/A"; fi

    if [ "$TIME_LAB4" != "N/A" ] && [ "$TIME_LAB5" != "N/A" ]; then
        SPEEDUP=$(echo "scale=2; $TIME_LAB4 / $TIME_LAB5" | bc 2>/dev/null)
        SPEEDUP="${SPEEDUP}x"
    else
        SPEEDUP="N/A"
    fi

    printf "%-10s | %-15s | %-15s | %-10s\n" "Teste $i" "$TIME_LAB4" "$TIME_LAB5" "$SPEEDUP"
done

echo "============================================================"
```

Para o cálculo do tempo de execução dos labs 04 e 05, é feita a passagem dos nomes dos arquivos de entrada e o gabarito correspondente.

Como o código anterior do 04 não imprimia o tempo no código, foi passado os argumentos para o programa 4 usando o comando time.

Para o 05, é executado os labs passando os argumentos e filtrando o tempo de execução de cada.

Por fim, dividido os valores de tempo dos labs para ter uma scala de quão otimizado foi.

2.5 Resultados:

```
ana-carolina@Carabina ~/L/L/arquivos_atividade_extra_classe_5 (main)> ./benchmark.sh
============================================================
    BENCHMARK COMPARATIVO: LAB 4 vs LAB 5 (Otimizado)
============================================================
Teste      | Lab 4 (s)       | Lab 5 (s)       | Speedup
------------------------------------------------------------
Teste 1    | 0.001           | 0.000212        | 4.71x
Teste 2    | 0.002           | 0.000547        | 3.65x
Teste 3    | 3.888           | 0.544542        | 7.13x
============================================================
ana-carolina@Carabina ~/L/L/arquivos_atividade_extra_classe_5 (main)> _
```

3 Conversor .npy -> .emat: